Research Report — Task 1: Running and Interpreting the Original Multi-Agent System

Topic: *LLM-assisted PLC code generation and industrial automation programming*

Framework: Microsoft AutoGen (AgentChat API), `autogen-agentchat==0.7.5`

Model: `deepseek-chat` (OpenAI-compatible endpoint)

---

1 Introduction

The goal of Task 1 is to clone the repository

`catDforD/LLM_MAS_AutoGen`, get it running

locally, understand its logic, and extract / interpret its execution logs.

The repository implements a **technical-research assistant** in which several agents with

distinct roles cooperate to turn a topic into a short cited report. It ships two versions:

- **`basic.py`** — a fixed three-agent pipeline (`Planner -> Search -> Writer`).
- **`advanced.py`** — the same pipeline plus a `ReviewerAgent` and a one-revision feedback loop.

2 Environment Setup

- Python 3.13.5, `autogen-agentchat==0.7.5`, `autogen-ext[openai]==0.7.5`.
- A virtual environment was created on the D: drive and dependencies installed from `requirements.txt`.
- The LLM endpoint was configured in `.env` to use **DeepSeek** (`deepseek-chat`), which is

OpenAI-compatible and supports function calling. Connectivity was verified with `check_api.py`.

3 Architecture of the Original MAS

The two versions map cleanly onto the course concepts of *Agent*, *Harness*, and *MAS*:

Version	Agents	Team / routing	Tools
`basic.py`	Planner, Search, Writer	`RoundRobinGroupChat` (fixed order)	`search_literature`
`advanced.py`	Planner, Search, Writer, Reviewer	`SelectorGroupChat` + `choose_next` router	`search_literature`, `audit_citations`

Each agent has a single, clearly-scoped responsibility:

- **PlannerAgent** splits the topic into 2–3 concrete search questions and keywords.
- **SearchAgent** calls `search_literature` (a real tool that queries a bundled course source

library and the Crossref API) and reports the returned evidence with `[S#]` numbers.

- **WriterAgent** writes a 500–700 word Chinese report with `[S#]` citations.
- **ReviewerAgent** (advanced) checks whether the citations match the retrieved evidence and

requests at most one revision.

4 Run Results

4.1 Basic version (`basic.py`)

Running `basic.py` with the PLC topic produced a directory `outputs/basic_<timestamp>/` containing

`report.md`, `trace.jsonl`, `evidence_registry.json`, and `run_summary.json`.

The console run (see `screenshots/01_basic_run_console.png`) shows the three agents talking in order. The final report (`screenshots/04_basic_report.png`) is a well-structured Chinese report with five sections (problem definition, main findings, comparison, limitations, conclusion) and citations `[S1]`–`[S12]`.

4.2 Advanced version (`advanced.py`)

`advanced.py` produced `outputs/advanced_<timestamp>/` with the same files plus `review.md`.

The run (see `screenshots/02_advanced_run_console.png`) demonstrates the **generate–review–revise**

loop: the `ReviewerAgent` found concrete citation problems (for example, that several Crossref records with near-identical titles and no abstract could not support a claim about engineering practice), returned `REVISION_REQUIRED`, and the `WriterAgent` produced a revised draft. The final `review.md` (`screenshots/06_advanced_review.png`) ends with the deterministic citation audit (" 引用检查通过") confirming that all 15 cited sources were registered.

5 Execution-Log Interpretation

The richest artifact is `trace.jsonl`, which records every event in the conversation. The event types observed in a single run are:

Event type	Meaning
`TextMessage`	A normal chat message from an agent or the user.
`ToolCallRequestEvent`	An agent decided to call a tool, with arguments.
`ToolCallExecutionEvent`	The result returned by the tool.
`ToolCallSummaryMessage`	The tool result forwarded as a message (because the search agent disables reflection).
`ThoughtEvent`	The model's internal reasoning before emitting a message.
`TaskResult`	The final task result/termination record.

Three concrete observations from the basic run's trace:

1. **A real tool call with multiple queries.** `SearchAgent` issued **three** `search_literature`

calls with different query strings (e.g. one about "PLC code generation structured text ladder logic", another about "PLC code verification simulation human-in-the-loop functional safety"). Each returned an evidence list with `[S#]` numbers. This is the "tool call $\rightarrow$ evidence" link that

the course emphasizes.

1. **Message passing between agents.** The trace shows the planner's keywords flow into the search

agent's tool calls, whose summarized results then flow into the writer's report. The `ToolCallSummaryMessage` is how AutoGen passes tool output to the next agent without a second (unreliable) reflection call.

1. **Termination conditions.** The basic run stops when `WriterAgent` emits the sentinel

`BASIC_REPORT_DONE` (`TextMentionTermination`), with a `MaxMessageTermination` safety bound. The

advanced run uses `REVIEW_APPROVED` / `REVISION_REQUIRED` plus a max-message bound and a one-revision cap, which is why the loop cannot run forever.

1. **Evidence traceability.** `evidence_registry.json` maps every `[S#]` used in the report back to

its title, authors, year, and URL. The post-hoc `audit_citations` tool verifies that every cited `[S#]` was actually registered in the run, closing the reliability loop.

6 Conclusion

Task 1 was completed: the original MAS runs end-to-end on a local machine, its role separation and

routing are understood, and its execution logs (tool calls, agent-to-agent messages, termination, and citation evidence) were extracted and interpreted. The screenshots and log excerpts are included in `/screenshots`.